

ResumelQ

AI-Powered Resume Analyser — Architecture Documentation

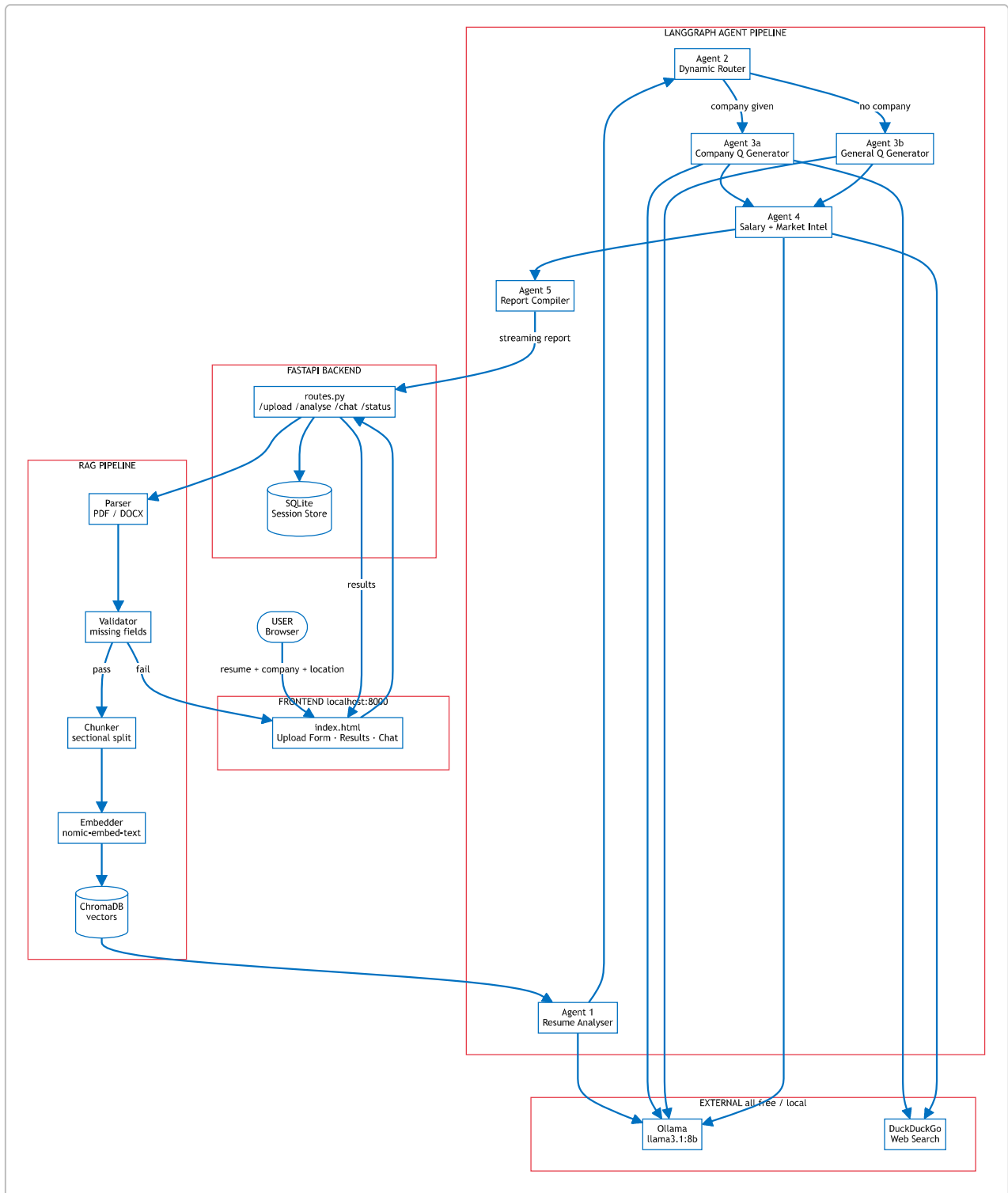
CONTENTS

- 1. High Level Architecture — End-to-end user flow through the system
- 2. Component Diagram — Internal modules and their connections
- 3. Database Schema — SQLite tables, columns, and relationships

Component	Technology	Purpose
Backend API	FastAPI + Uvicorn	REST endpoints, file upload, streaming
AI Orchestration	LangGraph 0.2+	Multi-agent state machine pipeline
LLM	Ollama — llama3.1:8b	Local inference, free, no API key
Embeddings	Ollama — nomic-embed-text	Resume chunk vectorisation
Vector DB	ChromaDB 0.6+	Stores and retrieves resume vectors
Metadata DB	SQLite	Session tracking, analysis results, chat
PDF Parsing	PyMuPDF	Extract text from PDF resumes
DOCX Parsing	python-docx	Extract text from Word resumes
Web Search	DuckDuckGo	Company info + salary data, no API key
Validation	Pydantic v2	Request/response schema validation

Diagram 1 — High Level Architecture

End-to-end flow: from user browser → FastAPI → RAG Pipeline → LangGraph Agents → Response



HOW IT WORKS — STEP BY STEP

- 1. Upload** — User uploads PDF/DOCX resume, enters company name (optional) and location. FastAPI saves a new session row in SQLite with status = *uploaded*.
- 2. Parse** — PyMuPDF (PDF) or python-docx (DOCX) extracts raw text from the file.
- 3. Validate** — Checks if mandatory fields (name, email, phone, experience) are present. If missing → UI prompts user to fill manually or re-upload. Status = *validation_failed*.
- 4. Chunk + Embed** — Resume split into sections (Contact / Experience / Skills / Education / Projects). Each

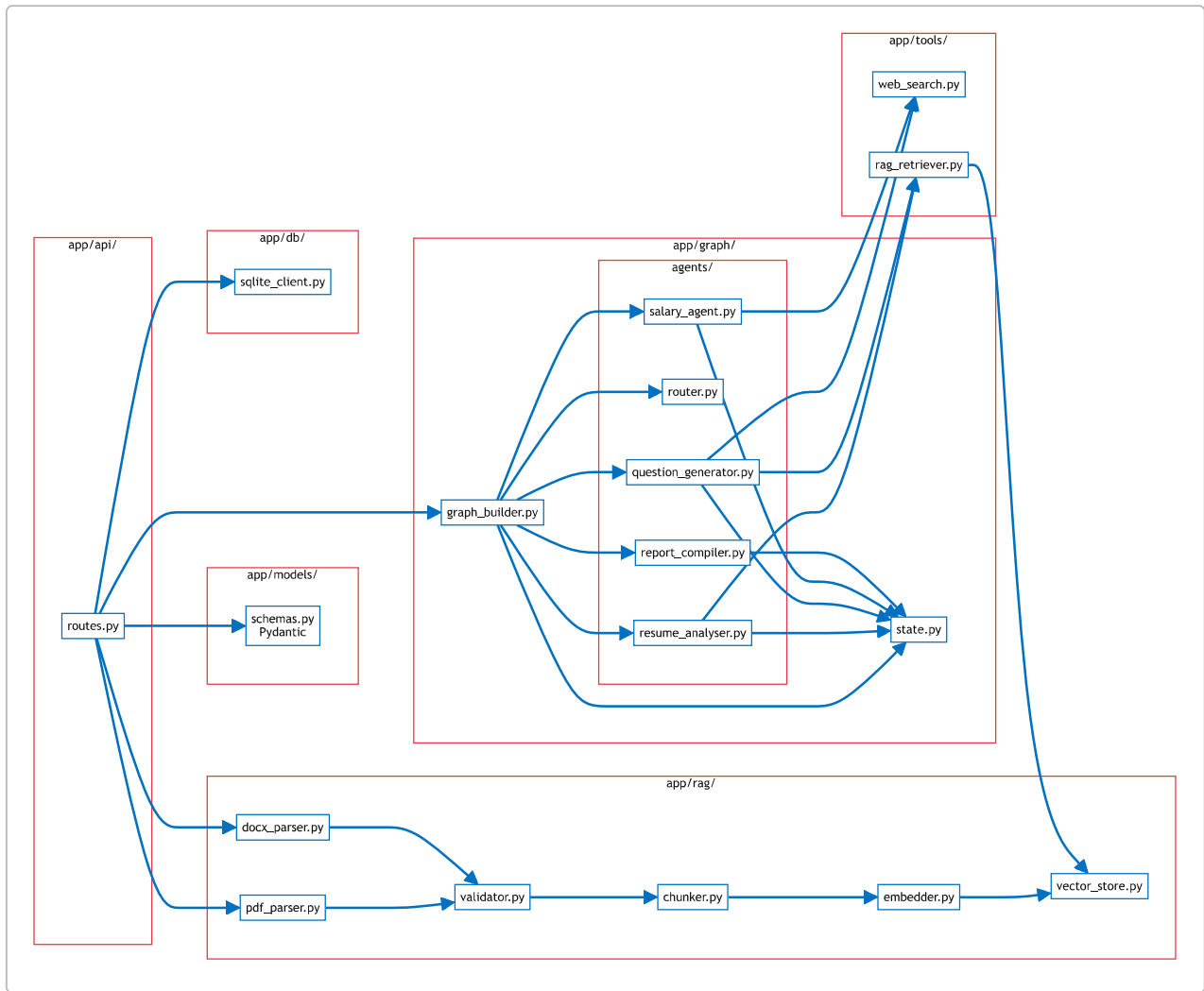
section converted to a 768-dim vector using Ollama nomic-embed-text and stored in ChromaDB. Status = *rag_ready*.

5. LangGraph starts — Agent 1 retrieves relevant chunks from ChromaDB and analyses skills, gaps, red flags. Agent 2 routes based on whether company name was given. Agent 3a/3b generates interview questions. Agent 4 searches DuckDuckGo for salary + hiring companies. Agent 5 compiles everything into a final markdown report.

6. Response — FastAPI streams the report to the browser. User can then ask follow-up questions in the chat interface.

Diagram 2 — Component Diagram

Internal Python modules, their single responsibility, and how data flows between them



MODULE RESPONSIBILITIES

routes.py

Entry point for all HTTP requests. Calls RAG pipeline, triggers LangGraph, reads/writes SQLite.

schemas.py

Pydantic models defining exact shape of every API request and response. FastAPI validates automatically.

pdf_parser.py / docx_parser.py

Extracts plain text from uploaded files. PyMuPDF for PDF, python-docx for Word files.

validator.py

Checks extracted text for mandatory fields — name, email, phone, experience years.

chunker.py

Splits resume text into labelled sections: Contact, Experience, Skills, Education, Projects.

embedder.py + vector_store.py

Converts each chunk into a 768-dim vector via Ollama, stores in ChromaDB with section metadata.

state.py

LangGraph ResumeState TypedDict — single shared object that all agents read from and write to.

graph_builder.py

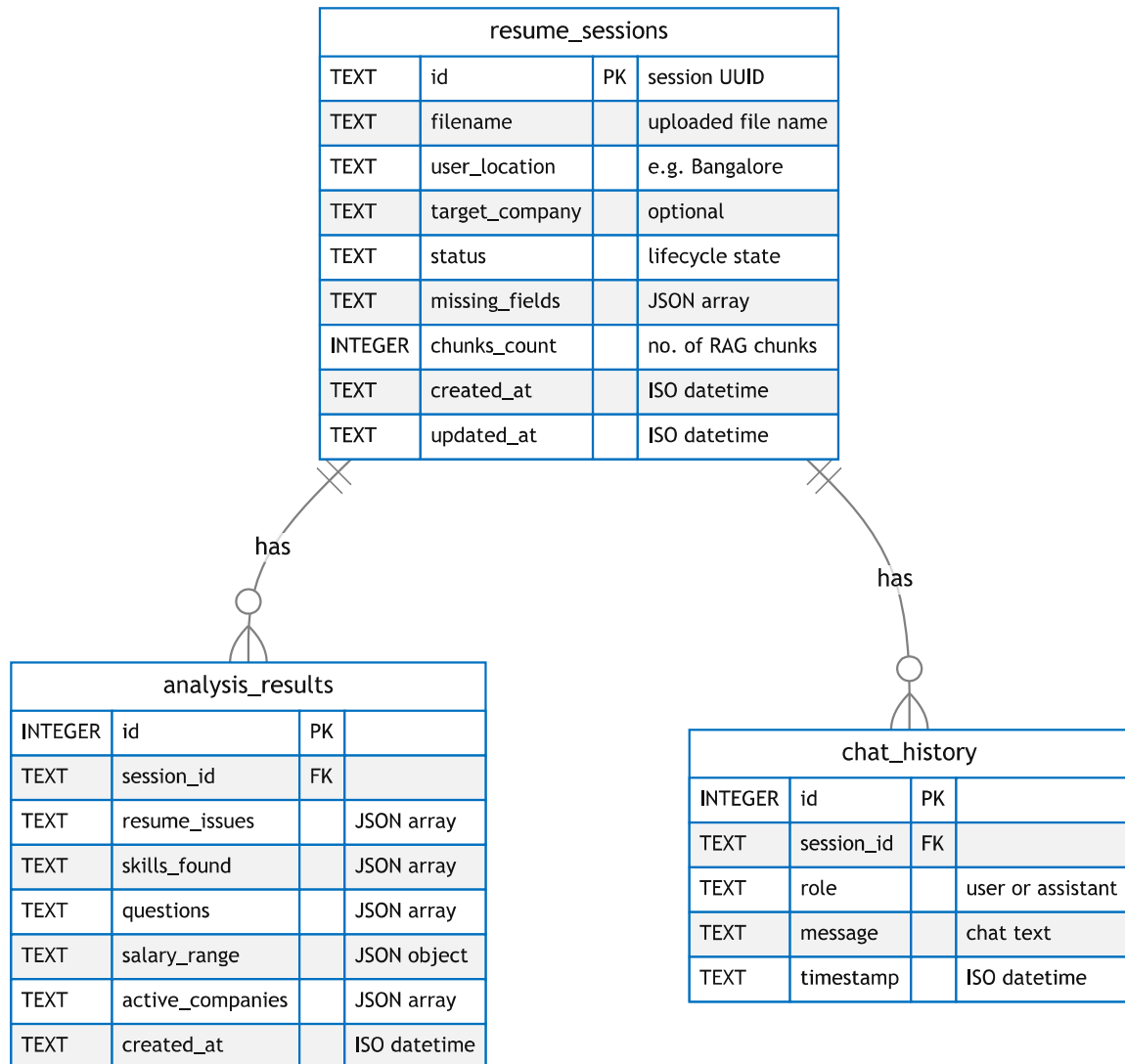
Compiles the LangGraph StateGraph — registers nodes, edges, conditional routing, and returns runnable.

web_search.py

sqlite_client.py

Diagram 3 — Database Schema (SQLite)

Three tables, their columns with data types, foreign key relationships, and status lifecycle



KEY DESIGN DECISIONS

status lifecycle

uploaded → rag_ready → processing → completed (or validation_failed / failed). Every agent transition updates this column so the frontend /status endpoint can show live progress.

JSON as TEXT columns

SQLite has no native array/object type. Lists and dicts are stored as JSON strings and parsed with `json.loads()` on read. Acceptable at this scale; use PostgreSQL JSONB for production.

One-to-many relationships

One session → many analysis_results rows (supports re-analysis). One session → many chat_history rows (full conversation stored in order by id ASC).

TEXT primary key for sessions

session_id is a UUID string (e.g. sess_a1b2c3) generated at upload time, not an auto-increment integer — makes it safe to share in API responses without exposing row counts.